

Representation-Aware Modularity with a Distance-Calibrated Null Model

Taejoon Han

Ulsan National Institute of Science and Technology
Ulsan, Republic of Korea
taejoon.han@unist.ac.kr

Junghoon Kim*

Ulsan National Institute of Science and Technology
Ulsan, Republic of Korea
junghoon.kim@unist.ac.kr

Dahee Kim

Ulsan National Institute of Science and Technology
Ulsan, Republic of Korea
dahee@unist.ac.kr

Taeho Ha

Soonchunhyang University
Asan, Republic of Korea
hth1690@sch.ac.kr

Abstract

Modularity is a standard objective for community detection, comparing observed intra-community connectivity with its null-model expectation. Modern node embeddings capture structural and semantic proximity beyond raw topology, but replacing adjacency with representation-induced affinity while retaining the classical degree-based null model makes the observed term geometry-dependent and the expected term geometry-agnostic. This creates a null-model mismatch: the objective may reward affinity already expected from representation proximity. This paper introduces RAM, a representation-aware modularity objective with a distance-calibrated null model. RAM preserves the observed-minus-expected principle by calibrating expected affinity with both degree heterogeneity and representation distance. We analyse its relation to classical modularity and develop divisive and multi-level greedy optimisers. Experiments on seven real-world datasets and synthetic scalability tests show that RAM improves over topology-only, direct embedding-clustering, and deep learning-based baselines, remains competitive under classical modularity while improving community quality on large graphs, and scales efficiently. Ablations confirm that both degree correction and distance calibration are essential.

CCS Concepts

• Information systems → Clustering.

Keywords

Community Detection, Modularity

ACM Reference Format:

Taejoon Han, Dahee Kim, Junghoon Kim, and Taeho Ha. 2026. Representation-Aware Modularity with a Distance-Calibrated Null Model. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26)*, November 07–11, 2026, Rome, Italy. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3799682.3840566>

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *CIKM '26, Rome, Italy*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2539-5/2026/11

<https://doi.org/10.1145/3799682.3840566>

1 INTRODUCTION

Community detection is a central problem in graph mining, data mining, and network science [24]. Given a graph, the goal is to identify groups of nodes, called communities, such that nodes within the same group are more densely connected to each other than to nodes in other groups [17]. Among the many formulations proposed for this task, modularity [33, 34] remains one of the most widely adopted. Classical modularity [34] measures the difference between the observed fraction of intra-community edges and the fraction expected under a null model. Its statistical interpretability has made modularity a standard and well-studied objective function for unsupervised community detection [17].

Recent advances in graph representation learning have made node representations a common input to graph mining tasks [20]. Such representations can be obtained from graph neural networks [27], random-walk embeddings [18, 36], spectral methods [3], or feature mappings derived from external data [19]. They capture information beyond raw adjacency, such as latent structural similarity, semantic proximity, and higher-order relational patterns [43]. However, using representations in modularity optimisation is not merely a matter of replacing edges with representation-induced affinities. Classical modularity is meaningful because its observed term and expected term are defined over the same structural signal. Once the observed affinity is derived from node representations, it becomes dependent on the geometry of the representation space. The degree-based null model of classical modularity does not account for this geometry. This raises a key question: how can modularity preserve its observed-minus-expected semantics when the observed affinity is representation-dependent?

A naive representation-aware extension of modularity replaces the adjacency term with representation-induced affinity while retaining the classical degree-based expectation. This formulation is not fully aligned with the observed-minus-expected interpretation of modularity. If two nodes are close in the representation space, high affinity may be expected even without evidence of community structure beyond their geometric proximity. Conversely, an observed edge between distant representations may carry a different meaning from an edge between nearby representations. A degree-only null model does not explicitly separate these effects. Consequently, the resulting objective may reward affinity that is already expected from representation geometry, rather than affinity that exceeds a proper null-model baseline. We refer to this issue as

a null-model mismatch in representation-aware modularity. The mismatch arises because the observed term is geometry-dependent, whereas the expected term remains geometry-agnostic. Therefore, representation-aware modularity motivates recalibrating the null model, rather than only redefining the observed affinity.

To address this mismatch, we propose *representation-aware modularity* (RAM), a modularity objective with a distance-calibrated null model. The key idea is to preserve the observed-minus-expected principle of classical modularity under representation-dependent affinity. RAM defines observed affinity on graph-supported node pairs and calibrates the expected term by both degree heterogeneity and representation distance. More specifically, the null model adjusts the classical degree-based expectation using a distance-dependent calibration function $g(d_{uv})$, where d_{uv} denotes the distance between the representations of nodes u and v . The function $g(\cdot)$ acts as the expected similarity conditioned on representation distance, not an arbitrary correction factor. As a result, RAM rewards within-community affinity only when it exceeds what is expected from both node degrees and representation distance.

This formulation positions RAM as an extension of modularity, rather than as an embedding-based clustering method. Under the special case where representation-induced affinity reduces to graph adjacency and $g(\cdot)$ is constant across distances, RAM reduces to classical modularity. Thus, RAM remains compatible with the classical objective while extending its null-model principle to representation-aware community detection.

Figure 1 summarises the RAM pipeline: representation-induced affinity defines the observed term, while the distance-calibrated null model defines the representation-aware expected term.

Optimising RAM is non-trivial. The objective inherits the combinatorial difficulty of modularity maximisation, and the calibrated expected term introduces additional computational challenges. To address this, we develop two strategies. First, **Betweenness-Guided Divisive Search Algorithm** (BGD) is a top-down approach that iteratively partitions the graph by removing high betweenness edges and evaluates each resulting partition using RAM. Second, **Multi-Level Greedy Aggregation Algorithm** (MGA) is a hierarchical bottom-up approach that alternates between node-level local moves and contracting communities into super-nodes.

The main contributions of this paper are as follows:

- We identify a null-model mismatch in representation-aware modularity and formulate RAM, a distance-calibrated objective with a representation-aware expected term.
- We preserve the observed-minus-expected principle by calibrating expected affinity with both degree heterogeneity and representation distance.
- We instantiate RAM with two optimisation strategies: a divisive search method and a Louvain-style multi-level optimiser with RAM-specific gain updates and centroid summaries.

2 REPRESENTATION-AWARE MODULARITY

In this section, we formally introduce representation-aware modularity (RAM), a modularity formulation that extends classical modularity by incorporating node representations while preserving the observed-minus-expected principle.

2.1 Preliminaries

Let $G = (V, E)$ be an undirected graph, where V denotes the set of nodes with $n = |V|$ and E denotes the set of edges with $m = |E|$. For a node $u \in V$, let $N(u) = \{v \in V \mid (u, v) \in E\}$ denote the set of its neighbours and $k_u = |N(u)|$ represent its degree.

Classical modularity. Community detection aims to partition V into a set of communities $C = \{C_1, \dots, C_k\}$. Classical modularity [34] evaluates a partition by comparing the observed intra-community connectivity against the expected connectivity under the null model:

$$Q(C) = \frac{1}{2m} \sum_{C \in C} \sum_{u, v \in C} \left(A_{uv} - \gamma \frac{k_u k_v}{2m} \right)$$

, where A_{uv} is the (u, v) entry of the adjacency matrix A , $\frac{k_u k_v}{2m}$ is the expected number of edges between u and v under the null model [34], and $\gamma > 0$ is a resolution parameter controlling the granularity of the resulting partition [37]. A partition is considered high-quality when the observed connectivity exceeds the expectation, yielding a large positive surplus.

Node representation. Each node $v \in V$ is associated with a d -dimensional vector $z_v \in \mathbb{R}^d$ in a representation space, collectively denoted by $\mathcal{Z} = \{z_v\}_{v \in V}$. These representations can be obtained from various sources, such as graph embedding models [18, 36], spectral methods [3], or feature mappings derived from external data [19]. Our framework treats the representations as input and is compatible with different representation learning pipelines.

Our primary objective is to extend the classical modularity framework to graphs equipped with node representations. To this end, we redefine the concept of intra-community surplus to account for both observed structural connectivity and geometric proximity in the representation space, while preserving the observed-minus-expected principle through a distance-calibrated null model.

2.2 Edge Affinity Based on Representations

Motivation. As discussed above, evaluating intra-community surplus requires considering both structural connectivity and geometric proximity. Relying solely on edge existence ignores the geometric proximity between nodes in the representation space. Therefore, we introduce a representation-induced edge affinity that assigns a weight to each observed edge based on the geometric proximity. This enables a more informative measure of intra-community connectivity that reflects both topology and representation.

Construction. To incorporate representation information while preserving the edge structure of the graph, we define a representation-induced affinity only on observed edges $(u, v) \in E$:

$$w_{uv} = A_{uv} \cdot \phi(z_u, z_v)$$

, where A_{uv} is the (u, v) entry of the adjacency matrix A , and $\phi(\cdot, \cdot)$ is a non-negative similarity function on the representation space.

We restrict affinity to observed edges by setting $w_{uv} = 0$ for $(u, v) \notin E$. This prevents node pairs from appearing connected solely based on their proximity in the representation space, keeping the observed affinity graph-supported. Let $2m_w = \sum_{u, v \in V} w_{uv}$ denote the total affinity mass of the graph. Unlike the structural edge mass $2m = \sum_{u, v} A_{uv}$, which depends purely on topology, $2m_w$ incorporates representation-induced affinities on observed edges.

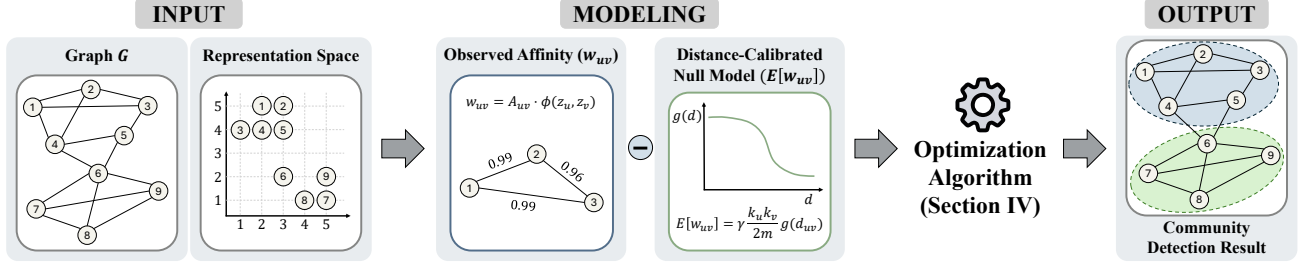


Figure 1: Overview of the proposed representation-aware modularity framework

2.3 Distance-Calibrated Null Model

Motivation. Classical modularity uses a degree-based null model, which is appropriate when community quality is evaluated purely from graph topology. In our framework, however, the observed affinity $w_{uv} = A_{uv}\phi(z_u, z_v)$ also depends on representation geometry, so a degree-only null model cannot capture geometry-induced variation in affinity.

Construction. To evaluate the expected affinity, the null model must account for structural and geometric factors. We factorise it into two components: a degree-based term inherited from classical modularity, and a distance-based term that captures the expected similarity induced by representation geometry. This product form should be understood as a degree-corrected calibration baseline, rather than an independence assumption between degree and representation geometry. The degree term $\frac{k_u k_v}{2m}$ determines the interaction opportunity assigned by the null model, while the calibration function $g(d_{uv})$, with $d_{uv} = \|z_u - z_v\|$, determines the expected affinity level at the corresponding representation distance. Combining the two, the expected affinity is defined as: $\mathbb{E}[w_{uv}] = \gamma \frac{k_u k_v}{2m} g(d_{uv})$, where $\gamma > 0$ is a standard resolution parameter.

Formally, $g(d_{uv})$ is the expected similarity at representation distance d_{uv} conditioned on the chosen similarity function ϕ :

$$g(d_{uv}) = \mathbb{E}[\phi(z_u, z_v) \mid \|z_u - z_v\| = d_{uv}, u \neq v]$$

The role of $g(\cdot)$ depends on the choice of similarity function ϕ . When ϕ captures directional information (e.g., cosine similarity), two node pairs at the same distance may have different similarity values depending on the directions of their embedding vectors. In this case, $g(\cdot)$ becomes a genuine conditional expectation: it averages out the directional variation and captures how similarity typically changes with distance.

Interpretation of the null term. Unlike classical modularity, the calibrated null mass in RAM is not required to sum to $2m_w$. The degree-based null model conserves total expected edge mass because node degrees are fixed and $\sum_u k_u = 2m$. By contrast, representation proximity is not a conserved quantity: many node pairs can simultaneously be close in the representation space. Thus, $\sum_{u,v} \mathbb{E}[w_{uv}]$ need not match the observed affinity mass. This reflects the role of $g(\cdot)$ as a distance-conditioned expected similarity, not a mass-preserving baseline.

2.4 Estimation of $g(d)$.

Computing $g(\cdot)$ exactly requires knowledge of the conditional distribution of ϕ , which is generally intractable in practice. We therefore estimate $g(\cdot)$ empirically from sampled node pairs.

We sample a set $\mathcal{S}$ of node pairs (u, v) uniformly at random with $u \neq v$, and compute their distances $d_{uv} = \|z_u - z_v\|$ and similarities $\phi(z_u, z_v)$. We partition the observed distance range into L bins $\{\mathcal{B}_1, \dots, \mathcal{B}_L\}$ with centres $\{c_1, \dots, c_L\}$, and for each bin compute the average similarity:

$$\hat{g}_\ell = \frac{1}{|\mathcal{B}_\ell|} \sum_{(u,v) \in \mathcal{B}_\ell} \phi(z_u, z_v)$$

Each $\hat{g}_\ell$ serves as the representative similarity at the bin centre c_ℓ . For any query distance d that falls between two adjacent bin centres c_ℓ and $c_{\ell+1}$, we estimate $g(d)$ by linearly interpolating $\hat{g}_\ell$ and $\hat{g}_{\ell+1}$. This yields a piecewise linear estimator $\hat{g}(d)$, which is $L_{\hat{g}}$ -Lipschitz, where $L_{\hat{g}}$ is its maximum slope.

2.5 Representation-Aware Modularity (RAM)

By combining the representation-induced edge affinity with the distance-calibrated null model, we define a new modularity formulation for graphs equipped with node representations.

DEFINITION 1 (REPRESENTATION-AWARE MODULARITY (RAM)). Given a graph $G = (V, E)$ with node representations $\mathcal{Z}$ and a partition $\mathcal{C} = \{C_1, \dots, C_k\}$ of V , the representation-aware modularity is defined as

$$Q_{\text{RA}}(\mathcal{C}) = \frac{1}{2m_w} \sum_{C \in \mathcal{C}} \sum_{u,v \in C} \left(w_{uv} - \gamma \frac{k_u k_v}{2m} g(d_{uv}) \right)$$

The inner summation ranges over all ordered node pairs within each community. Since the graph is undirected, each internal edge is counted twice, which correctly aligns with the global scaling factor $\frac{1}{2m_w}$. The product form in the expected term decomposes the null model into two factors: $k_u k_v / (2m)$ is the expected number of edges between u and v under the degree-based null model of classical modularity, while $g(d_{uv})$ captures the expected similarity at representation distance d_{uv} . This factorisation does not assume that topology and representation geometry are independent in the observed graph; rather, it defines a baseline determined by degree heterogeneity and distance-conditioned similarity, against which the observed graph-supported affinity is evaluated.

For notational convenience, let $p_{uv} = \gamma \frac{k_u k_v}{2m} g(d_{uv})$ denote the distance-calibrated null term. We define the observed and expected affinity sums of a community C as $w(C) = \sum_{u,v \in C} w_{uv}$ and $p(C) = \sum_{u,v \in C} p_{uv}$, respectively. $Q_{\text{RA}}(\mathcal{C})$ can be written equivalently as

$$Q_{\text{RA}}(\mathcal{C}) = \frac{1}{2m_w} \sum_{C \in \mathcal{C}} (w(C) - p(C))$$

Intuitively, $Q_{\text{RA}}(\mathcal{C})$ measures the surplus of observed affinity against the calibrated baseline. As a result, this formulation favours

intra-community affinity that exceeds a baseline determined by node degrees and representation distance.

Comparison with a naive representation-aware objective. A natural alternative is to replace graph adjacency by representation-induced affinity while keeping the degree-based expected term, scaled by a global average affinity. This leads to a naive representation-aware modularity:

$$Q_{\text{naive}}(C) = \frac{1}{2m} \sum_{C \in \mathcal{C}} \sum_{u,v \in C} \left[A_{uv} \cdot \phi(z_u, z_v) - \frac{k_u k_v}{2m} \bar{\phi} \right]$$

, where $\bar{\phi}$ denotes the global average affinity. Since the expected term is insensitive to representation distance, this objective may reward high-affinity edges between nearby representations even when such affinity is already expected because the two endpoints are close in representation space.

In contrast, RAM replaces the global average affinity $\bar{\phi}$ with the calibration function $g(\cdot)$. The difference is therefore not merely whether representations are used, but where they enter the observed-minus-expected principle: RAM uses representations in both terms, defining the observed affinity and calibrating the expected affinity. This allows the objective to reward affinity that exceeds what is expected at the corresponding representation distance.

The following property shows that RAM recovers classical modularity as a special case.

PROPERTY 1 (REDUCTION TO CLASSICAL MODULARITY). *If $\phi(z_u, z_v) = 1$ for all $(u, v) \in E$ and $g(d) = 1$ for all distances d , then RAM reduces to classical modularity with resolution parameter γ .*

PROOF. Under $\phi(z_u, z_v) = 1$ for all $(u, v) \in E$, we have $w_{uv} = A_{uv}$ for all $u, v \in V$. Consequently, the total affinity mass aligns with the structural edge mass: $2m_w = \sum_{u,v \in V} w_{uv} = \sum_{u,v \in V} A_{uv} = 2m$. Moreover, under $g(d) = 1$, the distance-calibrated null term simply becomes the degree-based expectation $\gamma \frac{k_u k_v}{2m}$. Substituting these directly into the definition of $Q_{\text{RA}}(C)$ yields

$$Q_{\text{RA}}(C) = \frac{1}{2m} \sum_{C \in \mathcal{C}} \sum_{u,v \in C} \left(A_{uv} - \gamma \frac{k_u k_v}{2m} \right)$$

, which is the classical modularity with resolution parameter γ . $\square$

3 PROBLEM STATEMENT

Having defined the RAM objective function, we formalise the associated optimisation problem. Given a graph and its node representations, our goal is to find a partition that maximises RAM. As in classical modularity, the number of communities is not specified in advance, but naturally emerges from the optimisation process.

PROBLEM DEFINITION 1 (REPRESENTATION-AWARE MODULARITY MAXIMISATION). *Given an undirected graph $G = (V, E)$ and node representations $\mathcal{Z}$, the objective is to find an optimal partition $C^* = \{C_1, \dots, C_k\}$ of V such that $C^* = \arg \max_C Q_{\text{RA}}(C)$. The partition must satisfy $C_i \cap C_j = \emptyset$ for $i \neq j$ and $\bigcup_{i=1}^k C_i = V$.*

Note that the number of communities k is not predefined and emerges from optimisation.

EXAMPLE 1. *Consider the toy graph in Figure 1. We compare $C_1 = \{\{1, \dots, 5\}, \{6, 7, 8, 9\}\}$ and $C_2 = \{\{1, \dots, 6\}, \{7, 8, 9\}\}$. With $\gamma = 1$, cosine similarity, and $\hat{g}$ estimated from all $\binom{9}{2} = 36$ pairs using $L = 5$*

equal-width bins, RAM gives $Q_{\text{RA}}(C_1) \approx 0.470$ and $Q_{\text{RA}}(C_2) \approx 0.391$. Although node 6 is near the first group in the representation space, adding it creates several intra-community non-edge pairs, such as $(1, 6)$, $(2, 6)$, and $(3, 6)$. These pairs contribute nothing to the observed affinity $w(C)$ because affinity is graph-supported, but they still increase the null term $p(C)$. Hence, the surplus $w(C) - p(C)$ decreases, and RAM favours the partition with stronger graph-supported affinity over the distance-calibrated baseline.

We next analyse two theoretical aspects of RAM: the computational complexity of its maximisation, and the stability of the objective under estimation error of the calibration function $g(\cdot)$.

THEOREM 1. *Representation-aware modularity maximisation problem is NP-hard.*

PROOF. We prove this by reduction from classical modularity maximisation, which is known to be NP-hard [8]. As established in Property 1, setting $\phi(z_u, z_v) = 1$, $g(d) = 1$, and $\gamma = 1$ reduces RAM exactly to the classical modularity formulation. Since classical modularity is a special case of Q_{RA} , it follows that maximising $Q_{\text{RA}}(C)$ is at least as hard, confirming its NP-hardness. $\square$

THEOREM 2 (INAPPROXIMABILITY OF RAM MAXIMISATION). *For any constant $\rho > 0$, unless $P = NP$, there is no polynomial-time algorithm that returns, for every instance with $Q_{\text{RA}}^* > 0$, a partition C satisfying $Q_{\text{RA}}(C) \geq \rho \cdot Q_{\text{RA}}^*$, where Q_{RA}^* denotes the optimal maximum value of RAM.*

PROOF. Since classical modularity maximisation is a special case of RAM by Property 1, any polynomial-time ρ -approximation algorithm for RAM yields a polynomial-time ρ -approximation algorithm for classical modularity maximisation, contradicting its inapproximability for any constant $\rho > 0$ unless $P = NP$ [13]. $\square$

PROPOSITION 1 (STABILITY UNDER ESTIMATION ERROR OF $g(d)$). *Let $Q_{\text{RA}}^{(g)}(C)$ and $Q_{\text{RA}}^{(\hat{g})}(C)$ denote the RAM objectives computed with g and $\hat{g}$, respectively. If $\sup_d |\hat{g}(d) - g(d)| \leq \varepsilon$, then for any partition $C = \{C_1, \dots, C_k\}$,*

$$|Q_{\text{RA}}^{(\hat{g})}(C) - Q_{\text{RA}}^{(g)}(C)| \leq \frac{\gamma \varepsilon}{4m m_w} \sum_{i=1}^k \text{vol}(C_i)^2$$

, where $\text{vol}(C_i) = \sum_{u \in C_i} k_u$. In particular,

$$|Q_{\text{RA}}^{(\hat{g})}(C) - Q_{\text{RA}}^{(g)}(C)| \leq \gamma \varepsilon \cdot \frac{m}{m_w}$$

PROOF. From the definition of RAM,

$$Q_{\text{RA}}^{(\hat{g})}(C) - Q_{\text{RA}}^{(g)}(C) = \frac{\gamma}{4m \cdot m_w} \sum_{i=1}^k \sum_{u,v \in C_i} k_u k_v (g(d_{uv}) - \hat{g}(d_{uv})).$$

Taking absolute values and using $|\hat{g}(d) - g(d)| \leq \varepsilon$ gives

$$|Q_{\text{RA}}^{(\hat{g})}(C) - Q_{\text{RA}}^{(g)}(C)| \leq \frac{\gamma \varepsilon}{4m m_w} \sum_{i=1}^k \sum_{u,v \in C_i} k_u k_v$$

Since $\sum_{u,v \in C_i} k_u k_v = \left(\sum_{u \in C_i} k_u \right)^2 = \text{vol}(C_i)^2$, the first bound follows. Also, $\sum_{i=1}^k \text{vol}(C_i)^2 \leq \left(\sum_{i=1}^k \text{vol}(C_i) \right)^2 = (2m)^2$, which yields

$$|Q_{\text{RA}}^{(\hat{g})}(C) - Q_{\text{RA}}^{(g)}(C)| \leq \gamma \varepsilon \cdot \frac{m}{m_w}$$

Therefore, the proposition is proved. $\square$

4 ALGORITHMS

Maximising RAM is challenging because its distance-calibrated null term depends on dense representation-distance interactions. We present BGD as a divisive reference optimiser and MGA as a scalable multi-level greedy optimiser with RAM-specific gain updates and centroid summaries.

4.1 Betweenness-Guided Divisive Search Algorithm (BGD)

As a reference optimiser for small and medium-sized graphs, we also consider a betweenness-guided divisive strategy inspired by Girvan–Newman [34]. BGD maintains a pruned working graph while evaluating every candidate partition using RAM on the original graph. Starting from the original graph, it repeatedly removes edges with high weighted betweenness. To incorporate representation-aware affinity into the decomposition process, we define the length of an observed edge (u, v) as $\ell_{uv} = 1/(\epsilon + w_{uv})$, where $\epsilon > 0$ is a small smoothing constant. Thus, paths are biased towards high-affinity edges, and low-affinity bridge edges are more likely to receive high betweenness and be removed earlier.

Whenever an edge removal splits a connected component, the connected components of the current working graph define a candidate partition. BGD keeps the best partition found over the sequence of splits and returns the one with the largest RAM value. The process continues until no further splits are possible. As a practical search convention, BGD reports the best non-trivial partition encountered after at least one split. The all-in-one partition is therefore not used as the returned solution, although it remains a valid partition under the objective definition.

The dominant cost of BGD is repeated weighted edge-betweenness computation. Exact Brandes computation costs $O(nm + n^2 \log n)$ per recomputation on a weighted graph. We approximate it by sampling x source nodes, reducing the cost to $O(x(m + n \log n))$ per recomputation; by default, $x = \sqrt{n}$. Since many recomputations may be needed before decomposition, BGD remains costly and is used mainly as a reference optimiser. Scalable optimisation on large graphs is handled by MGA.

4.2 Multi-Level Greedy Aggregation Algorithm (MGA)

Framework overview. To optimise RAM at scale, we propose MGA, a multi-level hierarchical framework. At each level, the SINGLELEVELGREEDY procedure visits each node and moves it to a neighbouring community whenever the move gain is positive. The resulting communities are then contracted into super-nodes to form a coarsened graph. This cycle of local refinement and graph contraction repeats until no merge occurs.

4.2.1 SINGLELEVELGREEDY.

The SINGLELEVELGREEDY performs a single greedy pass: each node is moved at most once if a neighbouring community yields a positive gain. The pass terminates when no further positive move exists.

Local gain under RAM. Consider moving a node u from its current community A to a candidate community B . Because Q_{RA} is defined as a sum over node pairs, this move only affects the observed affinity and distance-calibrated null terms directly involving u .

Algorithm 1: SingleLevelGreedy($G, \mathcal{Z}, \phi, \gamma, \hat{g}$)

```

1  $C^* \leftarrow \{\{v\} : v \in V\}$ 
2  $\text{initSummaries}(C^*)$ 
3  $\text{visited}[v] \leftarrow \text{false}$  for all  $v \in V$ 
4  $\text{improvement} \leftarrow \text{true}$ 
5 while  $\text{improvement}$  do
6    $\text{improvement} \leftarrow \text{false}$ 
7   foreach node  $u \in V$  with  $\text{visited}[u] = \text{false}$  do
8      $A \leftarrow C(u)$ 
9      $\mathcal{N}(u) \leftarrow \{C(v) \mid v \in N(u)\}$ 
10     $B \leftarrow \arg \max_{C \in \mathcal{N}(u)} \Delta Q(u : A \rightarrow C)$ 
11    if  $\Delta Q > 0$  then
12       $\text{move}(u, A, B, C^*)$ 
13       $\text{updateSummaries}(A, B)$ 
14       $\text{visited}[u] \leftarrow \text{true}$ 
15       $\text{improvement} \leftarrow \text{true}$ 
16 return  $C^*$ 

```

For the observed term, since w_{uv} is zero for non-adjacent nodes, only neighbours of u contribute: $w(u, C) = \sum_{v \in N(u) \cap C} w_{uv}$. For the null term, all node pairs involving u contribute: $p(u, C) = \sum_{v \in C} p_{uv}$.

Unlike classical modularity, where the gain depends only on the neighbours of u and a community-level degree sum, the RAM null term aggregates $g(d_{uv})$ over all pairs between u and the community. As a result, the local gain combines the graph-supported observed affinity with the community-aggregated null term. Combining these, the objective gain from moving u is $\Delta Q(u : A \rightarrow B) =$

$$\frac{1}{2m_w} \left([w(u, B) - p(u, B)] - [w(u, A \setminus \{u\}) - p(u, A \setminus \{u\})] \right)$$

Local search strategy. To efficiently find the best move for a node u , we only consider candidate communities that contain at least one of its neighbours. This restricted candidate set is defined as $\mathcal{N}(u) = \{C(v) \mid v \in N(u)\}$, where $C(v)$ denotes the community that node v belongs to. For each candidate community $B \in \mathcal{N}(u)$, we compute the objective gain $\Delta Q(u : A \rightarrow B)$. We then move u to the community that offers the highest positive gain ($\Delta Q > 0$). Otherwise, u stays in its current community A .

Centroid-based approximation. Evaluating $p(u, C) = \sum_{v \in C} p_{uv}$ exactly requires iterating over all nodes in C , which makes the per-move cost scale with community size. To reduce this cost, we approximate $p(u, C)$ using a community-level summary. For each community C , we maintain the degree sum $K_C = \sum_{v \in C} k_v$, the embedding sum $Z_C = \sum_{v \in C} z_v$, and the cardinality $|C|$. The centroid of C is $\bar{z}_C = Z_C/|C|$, and the null interaction between u and C is approximated as $\tilde{p}(u, C) = \gamma \frac{k_u K_C}{2m} \hat{g}(\|z_u - \bar{z}_C\|)$.

This replaces $|C|$ distance-dependent evaluations with a single evaluation of $\hat{g}$ at the node-centroid distance, reducing the null-term cost from $O(|C|d)$ to $O(d)$ per candidate community.

Whenever a node u moves from A to B , the three summaries are updated incrementally:

$$\begin{aligned} Z_A &\leftarrow Z_A - z_u, & K_A &\leftarrow K_A - k_u, & |A| &\leftarrow |A| - 1, \\ Z_B &\leftarrow Z_B + z_u, & K_B &\leftarrow K_B + k_u, & |B| &\leftarrow |B| + 1. \end{aligned}$$

Each update runs in $O(d)$ time and requires $O(kd)$ total memory, where k is the current number of communities. Combined with the local search over $\mathcal{N}(u)$, the cost of evaluating and executing a move becomes $O(|\mathcal{N}(u)| + |\mathcal{N}(u)|d)$.

We bound the error introduced by this approximation. Since the approximation replaces multiple distance evaluations with a single one at the centroid, its accuracy depends on how $\hat{g}(\cdot)$ varies with distance and how spread out the community members are around their centroid. We make this dependence precise as follows.

First, assume that $\hat{g}(\cdot)$ is $L_{\hat{g}}$ -Lipschitz, i.e., $|\hat{g}(x) - \hat{g}(y)| \leq L_{\hat{g}}|x - y|$ for all $x, y \geq 0$. Second, we measure how spread out community C is around its centroid via $\sigma_C = \frac{1}{K_C} \sum_{v \in C} k_v \|z_v - \bar{z}_C\|$, which is small when the members of C are tightly clustered around $\bar{z}_C$.

Combining these, the approximation error satisfies

$$\begin{aligned} |p(u, C) - \tilde{p}(u, C)| &= \left| \gamma \frac{k_u}{2m} \sum_{v \in C} k_v \hat{g}(\|z_u - z_v\|) - \gamma \frac{k_u K_C}{2m} \hat{g}(\|z_u - \bar{z}_C\|) \right| \\ &\leq \gamma \frac{k_u}{2m} \sum_{v \in C} k_v |\hat{g}(\|z_u - z_v\|) - \hat{g}(\|z_u - \bar{z}_C\|)| \\ &\leq \gamma \frac{k_u L_{\hat{g}}}{2m} \sum_{v \in C} k_v \left| \|z_u - z_v\| - \|z_u - \bar{z}_C\| \right| \\ &\leq \gamma \frac{k_u L_{\hat{g}}}{2m} \sum_{v \in C} k_v \|z_v - \bar{z}_C\| = \gamma \frac{k_u K_C}{2m} L_{\hat{g}} \sigma_C. \end{aligned}$$

The final bound has a clear interpretation: the approximation error is small when either $\hat{g}(\cdot)$ varies slowly with distance (small $L_{\hat{g}}$) or the community members are tightly clustered in the representation space (small σ_C). The latter condition is naturally aligned with RAM, which favours communities whose members are coherent in the representation space. This makes the centroid approximation a reasonable choice within the RAM framework.

Termination. Each accepted move has a positive estimated gain $\Delta Q > 0$. To avoid cyclic reassignments under the centroid-based approximation, we use a locked greedy pass in which each node is moved at most once. This caps the number of accepted moves by n , and the procedure terminates when no unlocked node has a positive-gain move to a neighbouring community. The SINGLELEVELGREEDY procedure is summarised in Algorithm 1.

4.2.2 Aggregation under RAM.

A key requirement in the multi-level design is that RAM remains consistent under graph contraction. Let X and Y denote two super-nodes, each representing a set of original nodes. We define the aggregated observed affinity and distance-calibrated null terms as

$$W_{XY} = \sum_{u \in X} \sum_{v \in Y} w_{uv}, \quad \text{and} \quad P_{XY} = \sum_{u \in X} \sum_{v \in Y} p_{uv}$$

Intuitively, W_{XY} sums the observed affinity of the original edges between X and Y , while P_{XY} sums the distance-calibrated null contributions over all node pairs with one endpoint in X and one in Y . If both quantities are computed exactly, evaluating Q_{RA} on the coarsened graph is algebraically equivalent to evaluating it on the original graph for the corresponding partition. This exact aggregation principle clarifies what must be preserved under contraction. In our scalable implementation, however, storing and updating all exact pairwise null interactions P_{XY} can be expensive. Therefore,

Algorithm 2: MGA($G, \mathcal{Z}, \phi, \gamma$)

```

1 computeAffinities( $G, \mathcal{Z}, \phi$ )
2  $\hat{g} \leftarrow$  estimateCalibration( $G, \mathcal{Z}, \phi$ )
3  $G' \leftarrow G$ 
4 while true do
5    $C \leftarrow$  SingleLevelGreedy( $G', \mathcal{Z}, \phi, \gamma, \hat{g}$ )
6   if  $|C| = |V'|$  then
7     break
8    $G' \leftarrow$  graphContract( $G', C$ )
9    $C^* \leftarrow$  projectToOriginal( $C$ )
10 return  $C^*$ 

```

MGA uses the centroid-based summaries introduced in Section 4.2.1 to approximate the null interactions at coarser levels.

4.2.3 Multi-Level Iteration.

Local gain at the super-node level. At each level, MGA applies the same gain formulation as the SINGLELEVELGREEDY, now at the super-node level. For a super-node X and a community C , we define the aggregated observed and distance-calibrated null terms as

$$W(X, C) = \sum_{Y \in C} W_{XY}, \quad \text{and} \quad P(X, C) = \sum_{Y \in C} P_{XY}$$

Using these, the gain from moving X from its current community A to a candidate community B is given by $\Delta Q(X : A \rightarrow B) =$

$$\frac{1}{2m_w} \left([W(X, B) - P(X, B)] - [W(X, A \setminus \{X\}) - P(X, A \setminus \{X\})] \right)$$

Algorithm description. At each level t , we apply SINGLELEVELGREEDY to the current graph $G^{(t)}$ to obtain a partition $C^{(t)}$. If no merges occur at this level, the algorithm terminates. Otherwise, each community in $C^{(t)}$ is contracted into a single super-node, yielding the coarsened graph $G^{(t+1)}$ with aggregated observed affinities W_{XY} and per-super-node summaries $(Z_X, K_X, |X|)$. These summaries are used to approximate the distance-calibrated null interactions at the coarser level, rather than explicitly materialising all exact pairwise terms P_{XY} . The procedure then continues on $G^{(t+1)}$ until termination, at which point the final super-node partition is mapped back to the original nodes. Note that, although we describe the procedure using indexed graphs $G^{(0)}, G^{(1)}, \dots$ for clarity, our implementation keeps only the current-level graph $G' = (V', E')$ in memory. The procedure is given in Algorithm 2.

Design rationale. MGA combines local refinement with multi-level aggregation. The SINGLELEVELGREEDY improves the partition through node-level reassignments, but local moves alone can trap the search in poor local optima and provide only a limited view of global structure. Graph contraction mitigates this limitation by enabling larger structural changes across coarsened levels. Unlike Louvain [7], however, aggregation in MGA must account for both observed affinity and distance-calibrated null interactions: while Louvain tracks only aggregated edge weights because its null model is degree-based, MGA additionally maintains per-super-node representation summaries to approximate the aggregated null interactions at each level.

Time complexity. Let L be the number of coarsening levels, and let (n_t, m_t) denote the number of nodes and edges at level t , and let α_t denote the number of SINGLELEVELGREEDY passes at level t .

With the centroid-based approximation, evaluating and executing a move for node u costs $O(|N(u)| + |N(u)|d)$ by scanning its neighbours once, where $N(u)$ is the set of neighbouring communities. Since $|N(u)| \leq |N(u)|$, one pass of SINGLELEVELGREEDY at level t costs $O(m_t d)$, and α_t passes cost $O(\alpha_t m_t d)$. Graph contraction, including the update of centroid summaries, costs $O(m_t + n_t d)$. Including the preprocessing cost $O(md + |S|d)$ for edge affinities and sampled-pair calibration function, the total running time of MGA is $O(md + |S|d + \sum_{t=0}^{L-1} (\alpha_t m_t d + m_t + n_t d))$. In practice, α_t is small and (n_t, m_t) decreases rapidly after coarsening, so the running time is dominated by the first few levels, which is consistent with the near-linear trend observed in EQ8.

5 EXPERIMENTS

5.1 Experimental Questions

To evaluate the proposed algorithm, we conduct extensive experiments to address the following experimental questions (EQs):

- **EQ1. Topology-based baselines:** How does the proposed model compare with topology-based community detection methods in effectiveness and efficiency?
- **EQ2. Embedding-based baselines:** How does the proposed model compare with clustering methods that use the same node representations?
- **EQ3. Deep learning-based baselines:** How does the proposed model compare with deep community detection methods, and can their embeddings further improve its effectiveness?
- **EQ4. Effect of γ :** How does the resolution parameter γ affect community detection quality?
- **EQ5. Distance calibration:** Does removing $\hat{g}(\cdot)$, the degree factor, or representation–graph alignment degrade performance?
- **EQ6. Centroid approximation:** How does centroid-based null-term approximation affect quality and running time?
- **EQ7. Robustness:** How robust is the model to different similarity functions and embedding methods?
- **EQ8. Scalability:** How does the algorithm scale in running time as the graph size increases?

5.2 Experimental Setting

Datasets. We use seven real-world datasets: Karate [52], Cora, Citeseer, and Amazon PC [16] have disjoint ground-truth communities, while DBLP, Amazon, and YouTube from SNAP [30] contain overlapping ground-truth communities. The datasets cover both small-scale attributed networks and large-scale real-world networks. Table 1 summarises their basic statistics, including the number of nodes ($|V|$), the number of edges ($|E|$), the number of clusters ($|C|$), and the average degree (d_{avg}).

Baselines. We compare MGA with three groups of baselines. The topology-based baselines mainly consist of Girvan–Newman [34], Greedy Modularity [11], Louvain [7], Leiden [44], and Infomap [38], covering classical divisive, agglomerative, modularity-based, and information-theoretic community detection methods. This selection follows a comparative analysis [29] reporting that modularity-based methods are among the top performers. The embedding-based baselines include k -means [32], spectral clustering [35], OPTICS [2], and HDBSCAN [9], all applied to the same node representations

Table 1: Real-world datasets

Type	Dataset	$ V $	$ E $	$ C $	d_{avg}
Small-scale	Karate	34	78	2	4.5
	Cora	2,708	10,556	7	7.7
	Citeseer	3,327	9,104	6	5.4
	Amazon PC	13,752	491,722	10	35.7
Large-scale	DBLP*	317,080	1,049,866	13,477	6.6
	Amazon*	334,863	925,872	75,149	5.5
	YouTube*	1,134,890	2,987,624	8,385	5.2

* Dataset with overlapping ground-truth communities.

used by MGA. We select the number of clusters for k -means via silhouette score [39] and for spectral clustering via the eigengap heuristic [46], and set the minimum samples for OPTICS to $\ln n$ [6]. The deep learning-based baselines include DMoN [45] and DGCluster [5]; we also evaluate DMoN+MGA and DGC+MGA by feeding their learned embeddings into MGA. For deep baselines, we use the authors' official implementations with default hyperparameters. Methods that do not finish within 24 hours are marked as OOT.

Experimental setup. For the similarity function ϕ , we use a rescaled cosine similarity $\phi(z_u, z_v) = (1 + \cos(z_u, z_v))/2$, mapping the cosine similarity to $[0, 1]$. The resolution parameter is set to $\gamma = 1$. For the calibration function $g(d)$, we estimate it from sampled node pairs. Specifically, we sample 30% of all possible node pairs uniformly at random, capped at a maximum of 5M pairs due to memory constraints. We then partition the sampled pairs by clustering their pairwise distances using k -means [32], where the number of clusters is automatically selected by maximising the silhouette score [39]. For datasets without node features, we initialise node representations using DeepWalk [36]. To ensure a fair comparison across datasets, we use AutoGL [53] with its default settings to automatically select unsupervised node embedding methods, and use the recommended embeddings as input to our model. BGD samples $\min(\sqrt{n}, 100)$ source nodes when estimating edge betweenness.

Evaluation metrics. For datasets with disjoint ground-truth communities, we report NMI [12] and ARI [23]. For Amazon, DBLP, and YouTube, whose ground-truth communities overlap, we report conductance [26] and modularity [34]. All scores are averaged over 10 independent runs.

5.3 Experimental Result

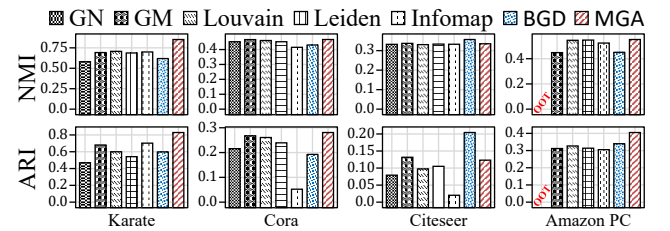


Figure 2: EQ1.1. Effectiveness on small-scale datasets

EQ1.1. Effectiveness on small-scale datasets. Figure 2 reports the NMI and ARI results on the four real-world datasets with disjoint ground-truth communities. GN did not finish on Amazon PC within the 24-hour time limit and is therefore marked as OOT. Overall, MGA achieves higher clustering quality than the topology-only

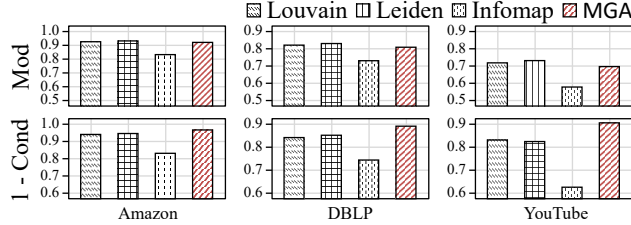
Table 2: EQ2. Performance comparison on small-scale datasets (AG+: baseline with AutoGL embeddings (same as MGA))

Dataset	NMI ($\uparrow$)					ARI ($\uparrow$)					Time (s) ($\downarrow$)				
	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB
Karate	0.8519	0.2260	0.8255	0.5434	0.2260	0.8289	0.1095	0.8027	0.5019	0.1095	0.0006	0.0920	0.0309	0.0323	0.0010
Cora	<u>0.4673</u>	0.5131	0.0249	0.3799	0.2127	<u>0.2808</u>	0.3730	0.0020	0.0025	-0.0106	<u>0.7452</u>	2.3741	3.4009	1.0971	0.5323
Citeseer	0.3351	0.1425	0.0758	<u>0.3563</u>	0.3593	0.1232	<u>0.0410</u>	0.0008	0.0032	0.0131	1.2368	5.9057	6.8476	2.1494	<u>1.8969</u>
Amazon PC	0.5523	0.2021	0.0033	<u>0.3298</u>	0.0495	0.4049	<u>0.1554</u>	-0.0005	0.0000	0.0000	2.4663	9.3387	318.0584	15.4501	<u>8.9653</u>

Table 3: EQ2. Performance comparison on large-scale datasets (AG+: baseline with AutoGL embeddings (same as MGA))

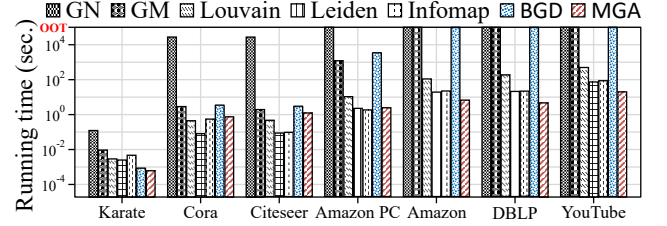
Dataset	1 - Conductance ($\uparrow$)					Modularity ($\uparrow$)					Time (s) ($\downarrow$)				
	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB
Amazon	0.9669	<u>0.8925</u>	OOT	0.1882	0.3466	0.9214	0.3725	OOT	0.2269	0.4325	6.7645	<u>284.2898</u>	OOT	12408.9564	19545.6532
DBLP	0.8915	<u>0.7558</u>	OOT	0.1320	0.2728	0.8091	<u>0.6808</u>	OOT	0.1540	0.2862	4.6996	<u>129.0685</u>	OOT	5664.4762	6210.7943
YouTube	0.9060	<u>0.6857</u>	OOT	OOT	OOT	0.6969	<u>0.0823</u>	OOT	OOT	OOT	20.0166	<u>685.6667</u>	OOT	OOT	OOT

baselines across the evaluated datasets, improving NMI and ARI by 1.05 $\times$ and 1.10 $\times$ on average, respectively. The improvement is most pronounced on Karate, where MGA obtains an NMI of 0.852 and an ARI of 0.829, exceeding the best baseline by 20.5% and 18.0%, respectively. On Citeseer, all methods exhibit relatively low performance, which is attributable to its fragmented graph structure: the graph contains 390 connected components, and only 64.7% of nodes belong to the largest component. Nevertheless, BGD and MGA remain competitive with, or outperform, the strongest baselines. These results indicate that representation-aware modularity can improve community alignment over topology-only objectives when node representations provide informative proximity signals.

**Figure 3: EQ1.2. Effectiveness on large-scale datasets (Mod: Modularity, 1 - Cond: 1 - conductance).**

EQ1.2. Effectiveness on large-scale datasets. Figure 3 reports modularity and 1 - conductance on the three large-scale datasets with overlapping ground-truth communities. Since the ground-truth overlaps whereas the evaluated methods return non-overlapping partitions, NMI and ARI are not directly applicable. We report conductance [26], which measures the normalised fraction of edges crossing community boundaries, and modularity, the canonical objective optimised by Louvain and Leiden. For consistency, we use 1 - conductance so that higher values indicate better separation. GN, GM, and BGD are excluded because they did not finish within the 24-hour time limit. On modularity, although MGA does not directly optimise classical modularity, it remains close to Leiden, the strongest modularity-optimising baseline, with an average gap of 0.022 across the three datasets, while achieving higher modularity than Infomap. On 1 - conductance, MGA consistently outperforms all baselines across the three datasets. The improvement is most pronounced on YouTube, where MGA obtains 0.906, exceeding the best baseline by 9.0%. These results indicate that MGA remains competitive under classical modularity while producing partitions with favourable boundary separation on large-scale networks.

EQ1.3. Efficiency. Figure 4 reports the running time of all algorithms across the seven datasets. For MGA, the reported time

**Figure 4: EQ1.3. Efficiency**

includes computing edge affinities w_{uv} , estimating $\hat{g}(d_{uv})$, and running the optimisation procedure; embedding learning time is excluded because node representations are treated as input to the proposed objective. On Cora and Citeseer, MGA is slightly slower than Louvain, Leiden, and Infomap, mainly because the precomputation of $\hat{g}(d_{uv})$ and w_{uv} accounts for over 98% of its total running time. The optimisation step itself takes less than 0.02 s, so the absolute overhead remains small. On the large-scale datasets, MGA shows favourable running time under the same reporting protocol, running 3.69 $\times$ faster than Leiden on average. This behaviour is consistent with its multi-level design, where graph contraction reduces the size of subsequent optimisation instances. Overall, the results suggest that distance calibration incurs manageable preprocessing overhead and that MGA remains practically scalable, although the reported times exclude embedding learning, which topology-only baselines do not require.

EQ2. Embedding-based baselines. We compare MGA with four embedding-based clustering baselines applied to the same node representations used as input to MGA. Tables 2 and 3 report effectiveness and efficiency on the small-scale and large-scale datasets, respectively. On the small-scale datasets, MGA achieves the best or second-best score in seven out of eight NMI/ARI cases. In contrast, every embedding-based baseline produces near-zero or negative ARI on at least one dataset, whereas MGA maintains non-trivial ARI across all four datasets. The advantage of MGA becomes more pronounced on the large-scale datasets. In terms of effectiveness, MGA achieves the best 1-conductance and modularity among the embedding-based baselines on all three large-scale datasets. In terms of efficiency, MGA is also consistently the fastest on large graphs, achieving an average speedup of over 34 $\times$ over k -means and orders-of-magnitude improvements over OPTICS and HDBSCAN on the datasets where they finish. These results indicate that directly applying conventional clustering to node representations is not sufficient to obtain reliable community partitions. The improvement of MGA is attributable not only to the use of embeddings, but

also to RAM, which couples representation geometry with graph topology through the distance-calibrated null model.

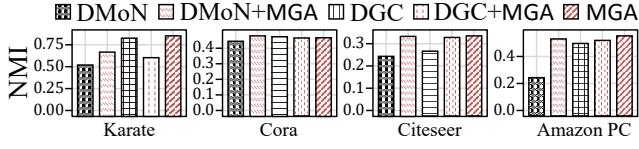


Figure 5: EQ3. Deep learning-based baselines

EQ3. Deep learning-based baselines. Figure 5 reports the NMI of two deep learning-based community detection baselines and their MGA-augmented variants on the four small-scale datasets. For each deep baseline, we conduct 10 independent runs and feed the learned embeddings into MGA to obtain the corresponding augmented variant. Across the four datasets, MGA achieves the best or near-best NMI, showing that the proposed objective remains competitive with deep learning-based community detection methods. Moreover, applying MGA on top of learned embeddings generally improves the corresponding baseline. For example, DMoN+MGA outperforms DMoN on all four datasets, improving NMI by 1.48× on average, with the largest gain on Amazon PC, where NMI more than doubles from 0.243 to 0.530. These results suggest that MGA can act as an effective representation-aware optimisation layer, rather than merely relying on the embedding quality itself.

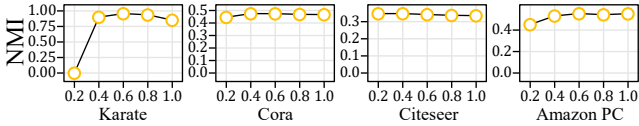


Figure 6: EQ4. Effect of γ

EQ4. Effect of γ . Figure 6 reports the NMI of MGA on the four small-scale datasets as the resolution parameter γ varies from 0.2 to 1.0. On Cora, Citeseer, and Amazon PC, the NMI increases in the small- γ regime and then stabilises around $\gamma = 0.4$, with $\gamma = 1$ remaining within 1–3.5% of the best observed NMI. Citeseer is particularly stable, with an NMI variation of less than 0.012 over the entire range of γ . Karate shows a sharper response: the NMI peaks at $\gamma = 0.6$ and decreases mildly toward $\gamma = 1$. This is expected on such a small graph, where changes in the null term can more directly affect the resulting partition. Even at $\gamma = 1$, however, MGA obtains an NMI of 0.852 on Karate, still outperforming all topology-only baselines in EQ1.1. Since classical modularity commonly adopts $\gamma = 1$ as the default resolution setting, we use the same convention instead of tuning γ separately for each dataset. These results show that MGA is not highly sensitive to γ around the default setting, supporting the use of $\gamma = 1$ in the main experiments.

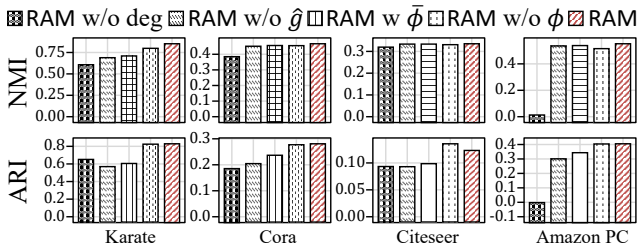


Figure 7: EQ5. Distance calibration

Table 4: EQ5. Ablation variants of RAM

Variant	Observed term	Null term
RAM w/o deg	$A_{uv} \cdot \phi(z_u, z_v)$	$\gamma \hat{g}(d_{uv})$
RAM w/o $\hat{g}$	$A_{uv} \cdot \phi(z_u, z_v)$	$\gamma \frac{k_u k_v}{2m}$
RAM w $\bar{\phi}$	$A_{uv} \cdot \phi(z_u, z_v)$	$\gamma \frac{k_u k_v}{2m} \bar{\phi}$
RAM w/o ϕ	A_{uv}	$\gamma \frac{k_u k_v}{2m} \hat{g}(d_{uv})$
RAM (full)	$A_{uv} \cdot \phi(z_u, z_v)$	$\gamma \frac{k_u k_v}{2m} \hat{g}(d_{uv})$

EQ5. Distance calibration. This experiment isolates the contribution of the calibration function $\hat{g}(d_{uv})$ in the RAM null term. We compare full RAM with four ablation variants in Table 4, all optimised using MGA, and report NMI and ARI in Figure 7. Across the four datasets, full RAM achieves the highest NMI and ARI in almost all cases. Replacing $\hat{g}(d_{uv})$ with a constant of 1 (RAM w/o $\hat{g}$) consistently underperforms using the global average affinity (RAM w $\bar{\phi}$), while full RAM performs best. This suggests that both the presence and the quality of the calibration function are important. Removing the degree term (RAM w/o deg) causes the sharpest degradation, collapsing on Amazon PC (NMI = 0.013, ARI = −0.002), which confirms the importance of degree heterogeneity. In contrast, removing ϕ from the observed term (RAM w/o ϕ) yields scores closer to full RAM than the null-term ablations, indicating that geometry-aware calibration mainly contributes through the expected term. Overall, the results support the need to calibrate the null model using both degree heterogeneity and distance-conditioned similarity.

Table 5: EQ6. Centroid approximation

Dataset	NMI ($\uparrow$)		ARI ($\uparrow$)		Time (s) ($\downarrow$)	
	Exact	Centroid	Exact	Centroid	Exact	Centroid
Karate	0.8509	0.8519	0.8340	0.8289	0.0006	0.0006
Cora	0.4686	0.4673	0.2777	0.2808	1.9659	0.7452
Citeseer	0.3380	0.3351	0.1305	0.1232	2.3083	1.2368
Amazon PC	0.5562	0.5523	0.4252	0.4049	48.4037	2.4663

EQ6. Centroid approximation. We compare two variants of MGA: *Exact*, which computes $p(u, C) = \sum_{v \in C} p_{uv}$ over all community members, and *Centroid*, which uses the centroid-based approximation $\tilde{p}(u, C)$. We restrict this comparison to small-scale datasets because exact evaluation is expensive on large graphs. Table 5 reports NMI, ARI, and running time on the four small-scale datasets. Across all datasets, *Centroid* achieves quality comparable to *Exact*, with average gaps of only 0.002 in NMI and 0.009 in ARI. At the same time, it substantially reduces running time, achieving an average speedup of 6.28×. This behaviour is consistent with the error bound in Section 4.2.1: when community members are coherent in the representation space, the spread σ_C remains small, thereby limiting the approximation error. Overall, the centroid approximation preserves clustering quality while improving efficiency, making it a practical choice for scalable MGA optimisation.

EQ7. Robustness. Figure 8 reports the NMI of MGA under different similarity functions and embedding methods on the four small-scale datasets. We first compare three similarity functions: rescaled cosine (default), Pearson correlation, and sigmoid inner product. Since RAM assumes non-negative affinities, we rescale Pearson correlation to $(1 + \rho)/2$, analogous to the cosine rescaling. On

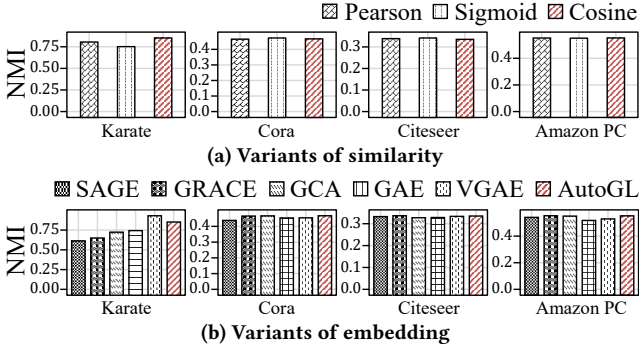


Figure 8: EQ7. Robustness

Cora, Citeseer, and Amazon PC, the three variants differ by less than 0.01 in NMI, indicating that MGA is reasonably robust to the choice of similarity function. We further test five unsupervised embedding methods: SAGE [19], GRACE [55], GCA [56], GAE [28], and VGAE [28]. For each method, we use reported dataset-specific hyperparameters when available, and default settings otherwise. On Cora, Citeseer, and Amazon PC, all embeddings yield NMI within 0.035 of AutoGL, suggesting robustness to the embedding source when embeddings are sufficiently well-conditioned. Karate shows a wider gap because default hyperparameters are less reliable on such a small graph. AutoGL avoids manual hyperparameter selection and consistently attains competitive NMI, supporting its use as the default embedding pipeline for MGA.

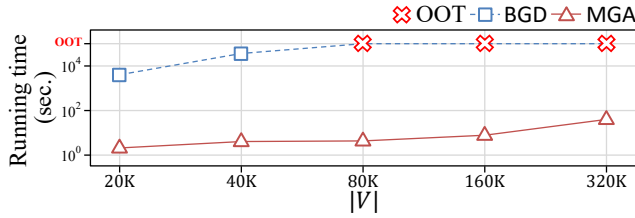


Figure 9: EQ8. Scalability

EQ8. Scalability. To evaluate scalability under controlled graph sizes, we generated synthetic attributed graphs using XMark [10] with its default generation parameters, varying $|V|$ from 20K to 320K by doubling. Figure 9 reports the total running time of BGD and MGA. BGD grows steeply with graph size, taking over 10 hours already at $|V| = 40K$ and exceeding the 24-hour time limit beyond that. This behaviour is consistent with the repeated weighted edge betweenness recomputation required by BGD, which incurs a super-linear cost and becomes impractical as the graph grows. In contrast, MGA scales smoothly across the entire range: increasing $|V|$ by $16\times$ ($20K \rightarrow 320K$) increases the running time by only about $19\times$, showing an approximately linear trend with respect to the generated graph size. This trend is consistent with the multi-level design of MGA, where graph contraction rapidly reduces the size of subsequent optimisation instances.

6 RELATED WORK

Modularity and non-standard null models. Classical modularity [34] compares observed connectivity with a degree-based null model. Its variants often modify the observed signal or null

model using spatial distance [14, 21, 41, 42], hypergraph interactions [25], or motif-enriched connectivity [49]. RAM is closest to null-model variants, but assigns a different role to distance. While spatial modularity uses external geographic coordinates to capture spatial dependence, compactness, or dispersion, RAM uses learned representation geometry to calibrate the expected affinity level. The factor $g(d_{uv})$ appears only in the expected term, whereas the observed term remains graph-supported as $A_{uv}\phi(z_u, z_v)$. Thus, RAM is not a direct spatial-modularity analogue, but a representation-aware null model for preserving observed-minus-expected semantics.

Attribute-aware and representation-based community detection. Attribute-aware community detection combines topology with node attributes, either by constructing structural–attribute similarities and partitioning an augmented graph [1, 4, 15, 48, 54], or by jointly factorising structural and attribute matrices to infer community assignments [22, 31, 47, 51]. Recent representation-based methods further integrate embeddings with community detection by reconstructing the modularity matrix [50], adding modularity-aware regularisation to graph autoencoders [40], directly optimising modularity through GNNs [45], or clustering learned embeddings under modularity-related objectives [5]. These methods use attributes or representations as evidence for assignment, training, or clustering. RAM follows a different paradigm: it treats node representations as input and asks how modularity should be recalibrated when representation-induced affinity is used as the observed signal. Representations therefore do not define a direct clustering space; they calibrate the expected affinity in the null model, so graph-supported affinity is rewarded only when it exceeds what is expected at the corresponding representation distance.

7 CONCLUSION

We introduced RAM, an objective for graphs with node representations that preserves observed-minus-expected semantics of classical modularity. The key idea is to calibrate the expected term using both degree heterogeneity and representation distance, evaluating representation-induced affinity against a distance-aware null baseline. We showed that RAM recovers classical modularity as a special case, inherits its hardness, and remains stable under bounded calibration error. Experiments and ablations confirm that distance calibration improves community detection quality and that both calibration components are essential, establishing RAM as an objective-level framework for representation-aware community detection beyond direct embedding-space clustering.

Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2020-II201336, Artificial Intelligence Graduate School Program(UNIST)) and (No. RS-2026-25527734, [Hyper-scale Industrial AI Research Support (R&D) Program], Development of industry-specified multimodal hyperscale foundation model based technologies for manufacturing, predictive maintenance, and process optimization), and the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2025-00523578, RS-2026-25468971)

GenAI Usage Disclosure

Generative AI tools were employed solely for writing assistance and code optimization. Specifically, they were used to improve the grammatical flow of the manuscript and to enhance the readability and efficiency of the implementation code. All core concepts, methodologies, and experimental results were developed independently by the authors, who maintain full responsibility for the content and integrity of this work.

References

- [1] Esmaeil Alnezhad, Babak Teimourpour, Mohammad Mehdi Sepehri, and Mehrdad Kargari. 2020. Community detection in attributed networks considering both structural and attribute similarities: two mathematical programming approaches. *Neural Computing and Applications* 32, 8 (2020), 3203–3220.
- [2] Mihael Ankerst, Markus M Breunig, Hans-Peter Kriegel, and Jörg Sander. 1999. OPTICS: Ordering points to identify the clustering structure. *ACM Sigmod record* 28, 2 (1999), 49–60.
- [3] Mikhail Belkin and Partha Niyogi. 2003. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural computation* 15, 6 (2003), 1373–1396.
- [4] Kamal Berahmand, Sogol Haghighi, Mehrdad Rostami, and Yuefeng Li. 2022. A new attributed graph clustering by using label propagation in complex networks. *Journal of King Saud University-Computer and Information Sciences* 34, 5 (2022), 1869–1883.
- [5] Aritra Bhowmick, Mert Kusan, Zexi Huang, Ambuj Singh, and Sourav Medya. 2024. Dgcluster: A neural framework for attributed graph clustering via modularity maximization. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 11069–11077.
- [6] Derya Birant and Alp Kut. 2007. ST-DBSCAN: An algorithm for clustering spatial-temporal data. *Data & knowledge engineering* 60, 1 (2007), 208–221.
- [7] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment* 2008, 10 (2008), P10008.
- [8] Ulrik Brandes, Daniel Delling, Marco Gaertler, Robert Görke, Martin Hoefer, Zoran Nikoloski, and Dorothea Wagner. 2006. Maximizing modularity is hard. *arXiv preprint physics/0608255* (2006).
- [9] Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. 2013. Density-based clustering based on hierarchical density estimates. In *Pacific-Asia conference on knowledge discovery and data mining*. Springer, 160–172.
- [10] Salvatore Citraro and Giulio Rossetti. 2021. X-Mark: a benchmark for node-attributed community discovery algorithms. *Social Network Analysis and Mining* 11, 1 (2021), 99.
- [11] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. 2004. Finding community structure in very large networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 70, 6 (2004), 066111.
- [12] Leon Danon, Albert Diaz-Guilera, Jordi Duch, and Alex Arenas. 2005. Comparing community structure identification. *Journal of statistical mechanics: Theory and experiment* 2005, 09 (2005), P09008–P09008.
- [13] Thang N Dinh, Xiang Li, and My T Thai. 2015. Network clustering via maximizing modularity: Approximation algorithms and theoretical limits. In *2015 IEEE International Conference on Data Mining*. IEEE, 101–110.
- [14] Paul Expert, Tim S Evans, Vincent D Blondel, and Renaud Lambiotte. 2011. Uncovering space-independent communities in spatial networks. *Proceedings of the National Academy of Sciences* 108, 19 (2011), 7663–7668.
- [15] Issam Falihi, Nistor Grozavu, Rushed Kanawati, and Younès Bennani. 2017. Anca: Attributed network clustering algorithm. In *International Conference on Complex Networks and their Applications*. Springer, 241–252.
- [16] Matthias Fey and Jan Eric Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. *arXiv:1903.02428 [cs.LG]* <https://arxiv.org/abs/1903.02428>
- [17] Santo Fortunato. 2010. Community detection in graphs. *Physics reports* 486, 3–5 (2010), 75–174.
- [18] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864.
- [19] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [20] William L Hamilton. 2020. *Graph representation learning*. Morgan & Claypool Publishers.
- [21] Joseph Hannigan, Guillermo Hernandez, Richard M Medina, Patrick Roos, and Paulo Shakarian. 2013. Mining for Spatially-Near Communities in Geo-Located Social Networks. In *AAAI Fall Symposium*.
- [22] Jin Huang, Tinghua Zhang, Weihao Yu, Jia Zhu, and Ercong Cai. 2021. Community detection based on modularized deep nonnegative matrix factorization. *International Journal of Pattern Recognition and Artificial Intelligence* 35, 02 (2021), 2159006.
- [23] Lawrence Hubert and Phipps Arabie. 1985. Comparing partitions. *Journal of classification* 2, 1 (1985), 193–218.
- [24] Di Jin, Zhizhi Yu, Pengfei Jiao, Shirui Pan, Dongxiao He, Jia Wu, Philip S Yu, and Weixiong Zhang. 2021. A survey of community detection approaches: From statistical modeling to deep learning. *IEEE Transactions on knowledge and data engineering* 35, 2 (2021), 1149–1170.
- [25] Bogumił Kamiński, Valérie Poulin, Paweł Pralat, Przemysław Szufel, and François Théberge. 2019. Clustering via hypergraph modularity. *PLoS one* 14, 11 (2019), e0224307.
- [26] Ravi Kannan, Santosh Vempala, and Adrian Vetta. 2004. On clusterings: Good, bad and spectral. *Journal of the ACM (JACM)* 51, 3 (2004), 497–515.
- [27] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [28] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).
- [29] Andrea Lancichinetti and Santo Fortunato. 2009. Community detection algorithms: a comparative analysis. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 80, 5 (2009), 056117.
- [30] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [31] Zhen Li, Zhisong Pan, Guyu Hu, Guopeng Li, and Xingyu Zhou. 2017. Detecting semantic communities in social networks. *IEICE TRANSACTIONS on Fundamentals of Electronics, Communications and Computer Sciences* 100, 11 (2017), 2507–2512.
- [32] James B McQueen. 1967. Some methods of classification and analysis of multivariate observations. In *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.* 281–297.
- [33] Mark EJ Newman. 2006. Finding community structure in networks using the eigenvectors of matrices. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 74, 3 (2006), 036104.
- [34] Mark EJ Newman and Michelle Girvan. 2004. Finding and evaluating community structure in networks. *Physical review E* 69, 2 (2004), 026113.
- [35] Andrew Ng, Michael Jordan, and Yair Weiss. 2001. On spectral clustering: Analysis and an algorithm. *Advances in neural information processing systems* 14 (2001).
- [36] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 701–710.
- [37] Jörg Reichardt and Stefan Bornholdt. 2006. Statistical mechanics of community detection. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 74, 1 (2006), 016110.
- [38] Martin Rosvall and Carl T Bergstrom. 2008. Maps of random walks on complex networks reveal community structure. *Proceedings of the national academy of sciences* 105, 4 (2008), 1118–1123.
- [39] Peter J Rousseeuw. 1987. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics* 20 (1987), 53–65.
- [40] Guillaume Salha-Galvan, Johannes F Lutzeyer, George Dasoulas, Romain Hennequin, and Michalis Vazirgiannis. 2022. Modularity-aware graph autoencoders for joint community detection and link prediction. *Neural Networks* 153 (2022), 474–495.
- [41] Marta Sarzynska, Elizabeth A Leicht, Gerardo Chowell, and Mason A Porter. 2016. Null models for community detection in spatially embedded, temporal networks. *Journal of Complex Networks* 4, 3 (2016), 363–406.
- [42] Paulo Shakarian, Patrick Roos, Devon Callahan, and Cory Kirk. 2013. Mining for geographically disperse communities in social networks by leveraging distance modularity. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1402–1409.
- [43] Xing Su, Shan Xue, Fanzhen Liu, Jia Wu, Jian Yang, Chuan Zhou, Wenbin Hu, Cecile Paris, Surya Nepal, Di Jin, et al. 2022. A comprehensive survey on community detection with deep learning. *IEEE transactions on neural networks and learning systems* 35, 4 (2022), 4682–4702.
- [44] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific reports* 9, 1 (2019), 5233.
- [45] Anton Tsitsulin, John Palowitch, Bryan Perozzi, and Emmanuel Müller. 2023. Graph clustering with graph neural networks. *Journal of Machine Learning Research* 24, 127 (2023), 1–21.
- [46] Ulrike Von Luxburg. 2007. A tutorial on spectral clustering. *Statistics and computing* 17, 4 (2007), 395–416.
- [47] Xiao Wang, Di Jin, Xiaochun Cao, Liang Yang, and Weixiong Zhang. 2016. Semantic community identification in large attribute networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30.
- [48] Zhonggang Wu, Zhao Lu, and Shan-Yuan Ho. 2016. Community detection with topological structure and attributes in information networks. *ACM Transactions on Intelligent Systems and Technology (TIST)* 8, 2 (2016), 1–17.
- [49] Jing Xiao, Ya-Wei Wei, Jing Cao, and Xiao-Ke Xu. 2024. Higher Order Fuzzy Membership in Motif Modularity Optimization. *IEEE Transactions on Fuzzy*

- Systems* 32, 12 (2024), 7143–7156.
- [50] Liang Yang, Xiaochun Cao, Dongxiao He, Chuan Wang, Xiao Wang, and Weixiong Zhang. 2016. Modularity based community detection with deep learning.. In *IJCAI*, Vol. 16. 2252–2258.
 - [51] Shuang Yang and Bo Yang. 2018. Enhanced network embedding with text information. In *2018 24th International Conference on Pattern Recognition (ICPR)*. IEEE, 326–331.
 - [52] Wayne W Zachary. 1977. An information flow model for conflict and fission in small groups. *Journal of anthropological research* 33, 4 (1977), 452–473.
 - [53] Ziwei Zhang, Yijian Qin, Zeyang Zhang, Chaoyu Guan, Jie Cai, Heng Chang, Jiyan Jiang, Haoyang Li, Zixin Sun, Beini Xie, et al. 2021. Autogl: A library for automated graph learning. *arXiv preprint arXiv:2104.04987* (2021).
 - [54] Yang Zhou, Hong Cheng, and Jeffrey Xu Yu. 2009. Graph clustering based on structural/attribute similarities. *Proceedings of the VLDB Endowment* 2, 1 (2009), 718–729.
 - [55] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep graph contrastive representation learning. *arXiv preprint arXiv:2006.04131* (2020).
 - [56] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2021. Graph contrastive learning with adaptive augmentation. In *Proceedings of the web conference 2021*. 2069–2080.